GitHub Copilot Extensions & Third-Party Agent Integration

Question: A site reliability engineering (SRE) team wants developers to query live application error traces from Sentry and inspect recent Kubernetes deployment statuses directly inside GitHub Copilot Chat (both in VS Code and on GitHub.com) using natural-language prompts like `@sentry list unresolved errors in shipping-service`. Which extensibility mechanism enables this workflow?

A. Adding Sentry API credentials inside `.github/copilot-instructions.md`

B. Installing and authorizing GitHub Copilot Extensions (GitHub Apps exposing custom Chat agents/skillsets) for the organization **[CORRECT ANSWER]**

C. Enabling the Public Code Duplication filter in Enterprise Copilot Policies

D. Creating a Copilot Enterprise Knowledge Base from exported Sentry CSV logs

Explanation & Exam Rationale

GitHub Copilot Extensions are specialized GitHub Apps that extend Copilot Chat with external tools, live telemetry, and domain-specific actions. Once installed from the GitHub Marketplace or built privately as custom agents/skillsets and approved by an organization administrator, developers invoke them with `@<extension-slug>` across VS Code, Visual Studio, and GitHub.com.

Extensibility Option	Underlying Technology	Governance & Capabilities
Copilot Extensions	GitHub Apps (Agents or Skillsets)	Queries live external APIs (Sentry, Datadog, Azure, Docker) using user OAuth permissions; controlled via Org Copilot Extension policy
Knowledge Bases	Indexed Markdown files in GitHub repos	Static/versioned documentation RAG only; cannot execute live external API calls
Built-in Participants	`@workspace`, `@terminal`, `@vscode`	Native IDE/workspace context only; no third-party SaaS integration

Key Exam Takeaway: Whenever a scenario requires interacting with *live external services or third-party developer tools* inside Copilot Chat via `@tool-name`, the answer is **GitHub Copilot Extensions** (governed by organization extension policies).